

Hackathon Unimar — Arquitetura e Plano de Análises

Guia sucinto para implementação do dashboard de fretes com Sisfrete/OpenSearch.

Sumário

1	Tech stack
2	Arquitetura
3	Como implementar
4	Análises importantes
4.1	Como implementá-las

1. Tech stack

Python como linguagem principal; **Streamlit** para o dashboard; **Requests** para comunicação HTTP com a API; **Pandas** para tratamento e análise; **Plotly** para visualizações interativas; **python-dotenv** para credenciais e configurações.

2. Arquitetura

Estrutura:

```
hackathon/  
■■■ app.py          # Streamlit / dashboard  
■■■ api.py          # Comunicação com Sisfrete  
■■■ queries.py      # Queries OpenSearch  
■■■ charts.py       # Gráficos Plotly  
■■■ .env            # Credenciais  
■■■ .env.example    # Modelo das variáveis  
■■■ .gitignore  
■■■ requirements.txt  
■■■ README.md
```

Arquivo	Responsabilidade
app.py	Interface Streamlit, filtros, orquestração e exibição.
api.py	HTTP, autenticação, POST, erros e retorno JSON.
queries.py	Queries OpenSearch e preparação dos resultados.
charts.py	Criação dos gráficos Plotly a partir dos dados.

Fluxo: app.py → queries.py → api.py → Sisfrete/OpenSearch → queries.py → charts.py → dashboard.

3. Como implementar

1. Criar o ambiente: `python -m venv .venv`.
2. Ativar o .venv e instalar `pip install -r requirements.txt`.
3. Criar `.env` a partir de `.env.example` e inserir as credenciais.
4. Implementar `api.py` para executar POSTs autenticados.
5. Criar em `queries.py` as consultas necessárias à análise.
6. Processar resultados com Pandas e gerar gráficos em `charts.py`.
7. Integrar tudo no Streamlit e executar: `streamlit run app.py`.

Configuração: manter `.env` fora do Git. Usar consultas agregadas (`size: 0`) quando possível e restringir índices/filtros para evitar consultas excessivamente amplas.

4. Análises importantes

Pergunta central sugerida: “Qual é o equilíbrio entre custo e prazo das transportadoras para cada destino?”

- 1. Custo × prazo por transportadora:** permite visualizar o trade-off entre preço e prazo.
- 2. Comparação para uma cidade:** custo médio, prazo médio e quantidade de cotações por transportadora.
- 3. Detalhamento da combinação:** mostrar transportadora, custo, prazo e ocorrência para o destino selecionado.

Campos principais: `nf.nome_cidade`, `nf.estado`, `nf.cotacoes.transportadora`, `nf.cotacoes.total`, `nf.cotacoes.frete`, `nf.cotacoes.p_min` e `nf.cotacoes.p_max`.

4.1. Como implementá-las

Filtro: o usuário seleciona cidade/estado no Streamlit.

Consulta: `queries.py` envia à API apenas os dados necessários.

Tratamento: Pandas organiza as cotações por transportadora e calcula métricas como média de custo, média de prazo e quantidade.

Visualização: Plotly apresenta custo × prazo e a comparação entre transportadoras.

Atenção: `nf.cotacoes` é um array de objetos não-nested. Agregações podem misturar campos de diferentes cotações do mesmo documento. Para relacionar precisamente uma transportadora ao seu próprio custo e prazo, extraia os documentos necessários e faça a correlação em Python.